OmniSight — Project Documentation & Technical Report

Field	Value
Project	OmniSight: Autonomous Market Intelligence Platform
Event	SerpApi India Hackathon 2026
Repository	https://github.com/shrikargs7-cloud/OmniSight
Document type	Project Documentation / Software Design Document
Version	1.0 (10 October 2026)

Table of Contents

1. Introduction
2. Problem Statement & Objectives
3. System Overview
4. Technology Stack
5. Project Structure (File-by-File)
6. Backend Layer — SerpApi Integration in Depth
7. Backend Layer — AI (Gemini) Routes
8. Data Object Model (Schemas)
9. API Contracts
10. Frontend Layer — State Model
11. Frontend Layer — Functions & Derived Metrics
12. Frontend Layer — Component / DOM Hierarchy
13. End-to-End Request Lifecycle
14. Error Handling & Resilience
15. Security & Configuration
16. Known Limitations (Estimated vs. Live Values)
17. Business Model
18. Future Roadmap
19. Setup & Deployment

1. Introduction

OmniSight is a web application that turns a single keyword (for example, *"electric scooters"*) into a market intelligence report. It calls **five SerpApi search engines at the same time**, combines their results into one data object, draws charts and graphs from that object, and sends a compressed version of it to **Google Gemini** to write a startup strategy and to answer chat questions.

This document describes every part of the system: each file, each function, each piece of state, the shape of every data object, and the exact SerpApi parameters and response fields used.

2. Problem Statement & Objectives

2.1 Problem

Validating a market idea normally means opening many tools separately:

- Google Trends to see if interest is growing or fading.
- Google Shopping to see what products sell for.
- Google Search to see what people are asking.
- Google News to check for bad press or recalls.
- YouTube to see what reviewers are saying.

Each tool has its own format, and none of them tells you what to *do* with the data.

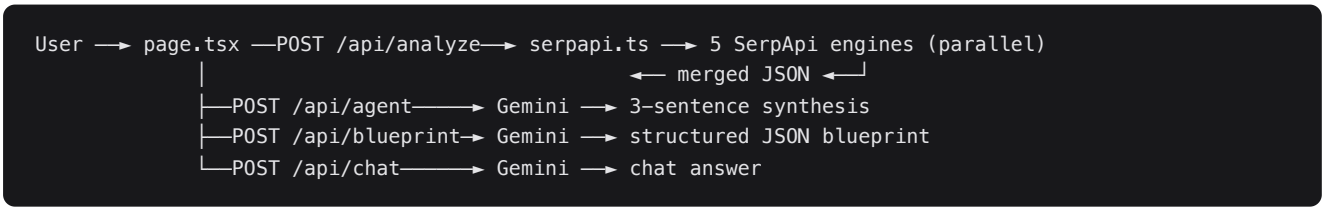
2.2 Objectives

#	Objective	How OmniSight meets it
O1	Gather multi-source market data in one action	One button triggers 5 SerpApi engines in parallel
O2	Present data visually	Area, bar, radar, and pie charts plus an interactive node graph
O3	Turn data into a plan	Gemini generates a structured JSON blueprint
O4	Let users ask follow-up questions	Interactive chat backed by `/api/chat`
O5	Never break during a demo	Every AI route has a fallback response

3. System Overview

OmniSight has three layers:

Layer	Responsibility	Files
Data layer	Calls SerpApi engines and returns raw JSON	`src/lib/serpapi.ts`
API layer	Next.js server routes that combine data and call Gemini	`src/app/api/*/route.ts`
Presentation layer	One React client page with 5 tabs, modals, and a chat sidebar	`src/app/page.tsx`



4. Technology Stack

Category	Technology	Version	Purpose
Framework	Next.js (App Router)	16.4.0	Pages and server API routes
UI library	React	19.3.0	Component rendering and state
Language	TypeScript	^5	Type safety
Styling	Tailwind CSS	^4	Dark-mode utility styling
Charts	Recharts	^3.10.1	Area, Bar, Radar, Pie charts
Graph	@xyflow/react (React Flow)	^12.12.0	SEO Threat Web node graph
Icons	lucide-react	^1.53.0	Interface icons
Search data	serpapi (official Node SDK)	^2.2.1	All search engine calls
AI	Google Gemini REST API	`gemini-1.5-flash`	Synthesis, blueprint, chat
Docs tooling	marked, puppeteer	^18 / ^25	Generating this PDF

5. Project Structure (File-by-File)

```
supply-chain-sleuth/
├── src/
│   ├── lib/
│   │   └── serpapi.ts           # 5 SerpApi engine wrapper functions
│   └── app/
│       ├── layout.tsx           # Root HTML layout and fonts
│       ├── globals.css          # Tailwind import and global styles
│       ├── page.tsx             # Entire client UI (tabs, charts, chat, modals)
│       └── api/
│           ├── analyze/route.ts  # Orchestrates the 5 SerpApi engines
│           ├── agent/route.ts    # Gemini 3-sentence market synthesis
│           ├── blueprint/route.ts# Gemini structured JSON startup blueprint
│           └── chat/route.ts     # Gemini conversational answers
├── public/
│   └── OmniSight_Hackathon_Report.pdf  # File served by "Export Report"
├── docs/                        # Report (MD/PDF) and pitch deck (MD/PPTX)
├── .env.local                   # SERPAPI_API_KEY, GEMINI_API_KEY (not committed)
├── package.json
└── README.md
```

File	Type	Runs on	Description
`src/lib/serpapi.ts`	Library	Server	Exports one async function per SerpApi engine. Each returns parsed JSON or `null` on failure.
`src/app/api/analyze/route.ts`	API route	Server	Accepts `{ query }`, runs all 5 engines with `Promise.all`, trims and reshapes the results.
`src/app/api/agent/route.ts`	API route	Server	Accepts `{ query, serpData }`, asks Gemini for a 3-sentence recommendation.
`src/app/api/blueprint/route.ts`	API route	Server	Accepts `{ query, serpData }`, asks Gemini for a strict JSON blueprint.
`src/app/api/chat/route.ts`	API route	Server	Accepts `{ message, serpData }`, asks Gemini for a short answer.
`src/app/page.tsx`	Client component	Browser	All UI, state, data transforms, and charts. Marked `"use client"`.
`src/app/layout.tsx`	Server component	Server	Wraps every page with `<html>` and `<body>`.

6. Backend Layer — SerpApi Integration in Depth

6.1 What SerpApi provides

SerpApi is a hosted API that runs a search on Google (and other engines) and returns the results page as structured JSON. OmniSight never scrapes HTML itself, never manages proxies, and never solves CAPTCHAs — SerpApi does all of that and returns clean fields such as `shopping_results`, `news_results`, and `related_questions`.

6.2 SDK usage

OmniSight uses the official `serpapi` npm package and its `getJson()` function:

```
import { getJson } from "serpapi";
const API_KEY = process.env.SERPAPI_API_KEY;
```

The key is read on the server only. It is never sent to the browser.

6.3 Engine reference

Engine 1 — Google Shopping

Item	Detail
Function	`searchGoogleShopping(query)`
Parameters	`engine: "google_shopping"`, `q: query`, `hl: "en"`, `gl: "us"`, `api_key`
Response field used	`shopping_results[]`
Sub-fields used	`title`, `source` (retailer), `price` (string, e.g. `"$129.99"`), `link`, `thumbnail`
Trimmed to	First 15 items → `marketPricing`
Used in	Retail Price bar chart, average/min/max price, wholesale estimate, B2B Wholesale Hub cards, pricing alert in chat, Gemini context
Missing key behaviour	Throws `"SERPAPI_API_KEY is missing"` (the only engine that does; it is treated as the required engine)

Engine 2 — Google News

Item	Detail
Function	`searchNews(query)`
Parameters	`engine: "google_news"` , `q: query` , `gl: "us"` , `api_key`
Response field used	`news_results[]`
Sub-fields used	`title` , `source.name` , `link` , `date`
Trimmed to	First 8 items → `sentimentNews`
Used in	"Live Data Feed" chat message, News Hype radar score, Gemini context (top 2–3 headlines)

Engine 3 — Google Trends

Item	Detail
Function	`searchGoogleTrends(query)`
Parameters	`engine: "google_trends"` , `q: query` , `data_type: "TIMESERIES"` , `date: "all"` (2004 to today), `api_key`
Response field used	`interest_over_time.timeline_data[]`
Sub-fields used	`date` , `values[0].extracted_value` (0–100 interest index)
Trimmed to	Full timeline → `historicalTrends`
Used in	Historical + forecast Area chart on the dashboard

Engine 4 — YouTube

Item	Detail
Function	`searchYouTube(query)`
Parameters	`engine: "youtube"` , `search_query: query` , `api_key` (note: YouTube uses `search_query` , not `q`)
Response field used	`video_results[]`
Sub-fields used	`title` , `link` , `channel.name` , `views` , `published_date`
Trimmed to	First 6 items → `influencerSentiment`
Used in	Media Monitor tab (embedded `<iframe>` players; video ID parsed from `link` after `v=`), Video Reach radar score

Engine 5 — Google Organic Search

Item	Detail
Function	`searchGoogleOrganic(query)`
Parameters	`engine: "google"` , `q: query` , `api_key`
Response fields used	`related_questions[]` ("People Also Ask"), `related_searches[]` , `organic_results[]`
Sub-fields used	`related_questions[].question` , `related_searches[].query` , `organic_results[].title/link`
Trimmed to	All questions, all searches, top 5 rankings → `seoData`
Used in	SEO Threat Web nodes, Demand radar score, Gemini context (top 5 questions)

6.4 Parallel orchestration

`/api/analyze` runs all five calls at once:

```
const [shoppingData, newsData, trendsData, youtubeData, organicData] =
  await Promise.all([
    searchGoogleShopping(query),
    searchNews(query),
    searchGoogleTrends(query),
    searchYouTube(query),
    searchGoogleOrganic(query)
  ]);
```

Total wait time equals the slowest single engine, not the sum of all five.

6.5 Failure isolation

Every wrapper has its own `try/catch` that returns `null`. The route then uses optional chaining with empty-array defaults, such as `newsData?.news_results || []`. If YouTube fails, the other four engines still render.

6.6 SerpApi credit usage

One "Execute" click = **5 SerpApi searches** (one per engine). Chat and blueprint calls reuse the data already fetched and do not use extra SerpApi credits.

---

7. Backend Layer — AI (Gemini) Routes

All three routes call: `POST https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=GEMINI_API_KEY`

Route	Input	Context sent to Gemini	Output	Fallback
`/api/agent`	`query`, `serpData`	Top 3 prices (source + price), top 2 headlines, top 3 questions	`{ response: string }` — 3 sentences, no markdown	Fixed arbitrage summary text
`/api/blueprint`	`query`, `serpData`	Top 4 prices, top 3 headlines, top 5 questions	Strict JSON (see §8.3)	Complete fallback blueprint object
`/api/chat`	`message`, `serpData`	Top 3 prices, top 2 headlines, user question	`{ response: string }` — 1–2 sentences	"Fallback mode" message

JSON enforcement (blueprint): the prompt gives Gemini the exact schema and says "RETURN ONLY JSON". The server removes any ```` ```json ```` fences and runs `JSON.parse`. If the text is empty or invalid, the fallback object is returned with HTTP 200 so the UI always renders.

---

8. Data Object Model (Schemas)

8.1 `AnalyzeResponse` (returned by `/api/analyze`, stored in `results`)

```
{
  query: string;
  marketPricing: ShoppingResult[];      // ≤ 15, from google_shopping
  sentimentNews: NewsResult[];          // ≤ 8,  from google_news
  historicalTrends: TimelinePoint[];    // full, from google_trends
  influencerSentiment: VideoResult[];   // ≤ 6,  from youtube
  seoData: {
    questions: { question: string }[];  // related_questions
    searches:  { query: string }[];     // related_searches
    rankings:  OrganicResult[];         // ≤ 5, organic_results
  };
}
```

8.2 Derived chart objects (built in the browser)

Object	Shape	Built by
`priceData`	`{ name, price: number, originalItem }[]` (≤ 8, price > 0)	`getPriceData()`
`trendsData`	`{ date: "YYYY-MM", Past_Present: number`	null, Future: number
`seoGraph`	`{ nodes: Node[], edges: Edge[] }`	`getSeoGraph()`
`radarData`	`{ subject, A: 0–100 }[]` (5 axes)	inline
`marketPieData`	`{ name, value, fill }[]` (4 slices)	inline constant
`chatMessages`	`{ type: "system"`	"news"

8.3 `AIBlueprint` (returned by `/api/blueprint`, stored in `aiBlueprint`)

```
{
  opportunityTitle: string;
  opportunityText: string;
  problemText: string;
  solutionText: string;
  goingWell: string[];
  needsAttention: string[];
  theWhy: string[];
  recommendation: string;
  detailedAnalysis: string;
  phase1: string[];   // 0–3 months
  phase2: string[];   // 3–6 months
  phase3: string[];   // 6–12 months
}
```

8.4 `ChatMessage`

```
{ type: "system" | "news" | "alert" | "llm" | "user"; text: string }
```

9. API Contracts

Method & Path	Request body	Success response	Error response
`POST /api/analyze`	`{ "query": "trimmers" }`	`AnalyzeResponse` (200)	`{ error }` 400 if no query, 500 on server error
`POST /api/agent`	`{ "query", "serpData" }`	`{ "response": "..." }`	`{ response: "Fatal Error..." }` 500
`POST /api/blueprint`	`{ "query", "serpData" }`	`AIBlueprint` (200, live or fallback)	`{ error }` 400 if no key, 500 on bad request
`POST /api/chat`	`{ "message", "serpData" }`	`{ "response": "..." }`	`{ response: "Fatal Error..." }` 500
`POST /api/export-report`	`{ "query", "results", "aiBlueprint", "avgRetailPrice", "estWholesalePrice", "minPrice", "maxPrice" }`	Binary PDF stream (`application/pdf`)	`{ error }` 500 on compilation failure

10. Frontend Layer — State Model

All state lives in the `Home` component in `page.tsx`.

State variable	Type	Initial	Purpose
`query`	`string`	`""`	Search box text
`loading`	`boolean`	`false`	Shows spinner during `/api/analyze`
`results`	`AnalyzeResponse	null`	`null`
`activeTab`	`string`	`"dashboard"`	Which main tab is shown
`selectedSeoNode`	`object	null`	`null`
`activeModal`	`string	null`	`null`
`agentMessage`	`string	null`	`null`
`blueprintTab`	`string`	`"executive"`	Blueprint sub-tab: executive / gap / roadmap
`blueprintLoading`	`boolean`	`false`	Blueprint request in progress
`aiBlueprint`	`AIBlueprint	null`	`null`
`chatInput`	`string`	`""`	Chat text box
`isChatting`	`boolean`	`false`	Chat request in progress
`userChatHistory`	`ChatMessage[]`	`[]`	User questions and AI answers
`isExporting`	`boolean`	`false`	Generating and streaming the dynamic executive PDF report
`exportError`	`string	null`	`null`

11. Frontend Layer — Functions & Derived Metrics

11.1 Event handlers

Function	Trigger	What it does
`handleAnalyze()`	"Execute" button	Clears old state → calls `/api/analyze` → stores `results` → fires `/api/agent` and `/api/blueprint` in the background (not awaited)
`handleChatSubmit(e)`	Chat form submit	Adds user message → calls `/api/chat` with `results` → adds AI answer
`onNodeClick(_, node)`	Click on SEO node	Computes display metrics for that node and stores them in `selectedSeoNode`
`triggerExportReport()`	"Download Executive PDF"	POSTs live product metrics to `/api/export-report`, receives server-rendered PDF buffer, and triggers download
`triggerPrintReport()`	"Print / Save via Browser"	Opens dedicated print window containing full formatted dossier of the browsed product and invokes `window.print()`

11.2 Data transforms

`getPriceData()` — strips non-numeric characters from each `price` string (`"$1,299.00"` → `1299.00`), drops items priced at 0, shortens retailer names longer than 10 characters, keeps the first 8.

`getTrendsData()` — takes every 6th Trends point (about one per half-year for monthly data), converts dates to `YYYY-MM`, then adds a 3-point forecast:

- `avgChange = (last − fifthFromLast) / 5`
- `Future[i] = max(0, last + avgChange × i × 2)` for i = 1, 2, 3, each 6 months apart
- The last real point is copied into `Future` so the two lines connect.

`getSeoGraph()` — builds one root node (the query), up to 4 question nodes on the left (from "People Also Ask", dashed blue animated edges) and up to 4 keyword nodes on the right (from Related Searches, purple animated edges).

11.3 Derived metric formulas

Metric	Formula	Source
Average retail price	mean of `priceData.price`	Live (Shopping)
Retail ceiling / floor	max / min of `priceData.price`	Live (Shopping)
Estimated wholesale price	average retail × 0.35	Estimate (assumed 65% margin)
Radar — Demand	min(100, questions × 15 + 40)	Derived from live count
Radar — News Hype	min(100, news × 10 + 30)	Derived from live count
Radar — Video Reach	min(100, videos × 20 + 40)	Derived from live count
Radar — Margins / Competition	85 / 90	Fixed values
SEO node volume	label length × 1200 + 4500	Illustrative estimate
SEO node CPC	label length × 0.15	Illustrative estimate
SEO node difficulty	min(99, label length × 4)	Illustrative estimate
Market channel pie	42 / 28 / 18 / 12 %	Fixed illustrative split

12. Frontend Layer — Component / DOM Hierarchy

```
<div root: flex h-screen, bg #050505>
├── Modal overlay (shown when activeModal ≠ null)
│   ├── "export"   → Generating PDF Report (auto-download)
│   ├── "api"      → OmniSight API Access card
│   ├── "settings" → Region (gl) / Language (hl) / Plan
│   └── "analysis" → AI Analysis Report (aiBlueprint.detailedAnalysis)
├── <aside> Sidebar
│   ├── Logo: OMNI SIGHT / Enterprise Intelligence
│   ├── CORE ENGINES: Market Dashboard · SEO Threat Web · Media Monitor
│   ├── COMMERCE & LAUNCH: B2B Wholesale Hub · Startup Blueprint
│   └── PLATFORM: Export Report (PDF) · API Access · Settings
└── <main>
    ├── Header: search <input> + EXECUTE button
    └── Tab content
        ├── dashboard
        │   ├── AreaChart  — Historical + forecast interest (Trends)
        │   ├── BarChart   — Retail prices by retailer (Shopping)
        │   ├── RadarChart — Market threat matrix
        │   └── Chat sidebar — OMNISIGHT AI (messages + input form)
        ├── seo        — <ReactFlow> graph + Node Intelligence panel
        ├── media      — Grid of YouTube <iframe> embeds + news list
        ├── wholesale  — Supplier cards (retail, est. bulk price, contact)
        └── blueprint
            ├── Hero + sub-tab bar
            ├── executive — Opportunity / Problem / Solution
            ├── gap       — Going Well / Needs Attention / The Why + AI Recommendation
            └── roadmap   — Now (0–3) · Next (3–6) · Later (6–12 months)
```

13. End-to-End Request Lifecycle

1. User types a keyword and clicks **Execute**.
2. `handleAnalyze` resets state and sets `loading = true`.
3. Browser sends `POST /api/analyze { query }`.
4. Server runs 5 SerpApi engines in parallel (§6.4).
5. Server trims and reshapes results into `AnalyzeResponse` (§8.1).
6. Browser stores `results`; charts, SEO graph, media, and wholesale tabs render immediately.
7. In the background, the browser sends `results` to `/api/agent` and `/api/blueprint`.
8. Gemini replies (or the fallback is used); `agentMessage` and `aiBlueprint` update, and the chat and Blueprint tabs fill in.
9. The user can ask questions in the chat; each goes to `/api/chat` with the same `results`.

14. Error Handling & Resilience

Failure	Where handled	Result
One SerpApi engine fails	`serpapi.ts` try/catch → `null`	That section is empty; others render
SerpApi key missing	`searchGoogleShopping` throws	`/api/analyze` returns 500; UI shows error
Gemini key invalid / quota hit	All AI routes	Fallback text or object returned with 200
Gemini returns non-JSON	`/api/blueprint` JSON.parse in try/catch	Fallback blueprint
Network error in browser	`.catch()` on each fetch	Logged; UI stays usable
Missing price string	`item.price?.replace(...)	
Invalid Trends date	`isNaN(date)` check	Uses current date

15. Security & Configuration

Variable	Required	Used by
`SERPAPI_API_KEY`	Yes	`src/lib/serpapi.ts`
`GEMINI_API_KEY`	Recommended	`/api/agent`, `/api/blueprint`, `/api/chat`

- Both keys are read only in server routes, so they are never included in browser JavaScript.
- `.env.local` is listed in `.gitignore` and must not be committed.
- The "API Access" modal shows a sample key for demonstration; it is not a working credential.

16. Known Limitations (Estimated vs. Live Values)

For transparency, these values are **not** live data:

- **Wholesale price** is an estimate (35% of average retail), not a real supplier quote.
- **Supplier contact emails** in the B2B Wholesale Hub are generated from the retailer name for display.
- **SEO node volume, CPC, and difficulty** are calculated from label length to illustrate the interface; they are not real keyword-planner numbers.
- **Market channel pie chart** and the Margins/Competition radar axes use fixed values.
- **Trend forecast** is a simple linear extrapolation, not a statistical model.
- Region and language in Settings are fixed to `gl=us`, `hl=en` in the engine calls.

(Note: The "Export Report" feature dynamically compiles an executive market dossier specific to the searched product via `/api/export-report`, incorporating live retail pricing, press sentiment, consumer search intent, and Gemini strategy matrices.)

Everything else — prices, retailers, headlines, Trends timeline, YouTube videos, "People Also Ask" questions, related searches, and organic rankings — comes live from SerpApi on every search.

17. Business Model

Tier	Price	Includes
Free	$0	3 searches per month
Pro	$99 / month	Unlimited searches, PDF export, full Trends history
Enterprise	Custom	API access, team seats, custom engines

Target users: e-commerce sellers and dropshippers, SaaS founders, marketing and SEO agencies, early-stage investors.

---

18. Future Roadmap

1. Make Settings region/language pass `gl` and `hl` to every engine.
2. Replace estimated SEO metrics with real keyword data.
3. Generate a per-search PDF from live results.
4. Add more SerpApi engines (Amazon, Walmart, Google Maps for local suppliers).
5. Save searches and compare markets over time.

---

19. Setup & Deployment

```
 git clone https://github.com/shrikargs7-cloud/OmniSight.git
cd OmniSight
npm install
# create .env.local with SERPAPI_API_KEY and GEMINI_API_KEY
npm run dev        # http://localhost:3000
npm run build && npm start   # production
```

The app can be deployed to Vercel; add both environment variables in the project settings.

OmniSight — built for the SerpApi India Hackathon 2026.